

Smart Movie Recommendation System

Documentation Report

1. Project Description

This project is a smart Movie Recommendation System developed using C# Console Application and AI-based recommendation logic. The system allows users to register, login, browse movies, search for movies, rate movies, and receive personalized movie recommendations. The recommendation engine analyzes user preferences and movie ratings using: - Content-Based Filtering - Collaborative Filtering - Cosine Similarity calculations

2. OOP Concepts Used

- Encapsulation: Private fields and public properties secure data and apply validation rules.
 - Inheritance: User and Admin inherit from the Person class.
 - Abstraction: Recommendation systems were implemented using interfaces and abstract structures.
 - Polymorphism: Multiple recommendation strategies were implemented through interfaces.
 - Interfaces: IRecommendation, ISearch, and IDataManager were used for modular architecture.
-

3. AI Recommendation Logic

The recommendation engine uses two intelligent recommendation strategies: **1. Content-Based Filtering** Recommends movies based on: - Genres - Tags - Directors - Actors **2. Collaborative Filtering** Analyzes user similarities based on rating behaviors. The system uses Cosine Similarity: $\text{Similarity}(A,B) = \frac{(A \cdot B)}{(|A| \cdot |B|)}$ Recommendation scores are calculated using: - Genre match - User similarity - Movie popularity - User ratings

4. System Architecture

The project follows Clean Architecture principles by separating the system into independent modules: - Models/ - Interfaces/ - Services/ - Data/ - Utilities/ - Program.cs The application loads JSON data into memory during startup and synchronizes updates back into the files dynamically.

5. Team Responsibilities

- Member 1 — Authentication & User Management
 - Member 2 — Movie Management
 - Member 3 — Search System
 - Member 4 — Rating System
 - Member 5 — AI Recommendation Engine
 - Member 6 — Console UI & System Integration
-

6. Development Challenges

- Managing JSON file operations safely.
 - Preventing file corruption during write operations.
 - Handling cold-start recommendation problems for new users.
 - Optimizing LINQ performance for recommendation calculations.
 - Integrating multiple modules into a single scalable system.
-

7. Future Improvements

- Add SQLite database support.
 - Create GUI desktop application.
 - Add Watch Later and Favorites features.
 - Improve AI recommendation accuracy using ML.NET.
 - Add recommendation confidence scores and analytics.
-

Team Distribution Table

Member	Responsibility
Member 1	Authentication & User Management
Member 2	Movie Management
Member 3	Search System
Member 4	Rating System
Member 5	AI Recommendation Engine
Member 6	Console UI & System Integration